ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/333112457

Algorithms for the Unrelated Parallel Machine Scheduling Problem with Sequence Dependent Setup Times

Conference Paper · November 2018

DOI: 10.1109/SCCC.2018.8705164

CITATIONS
3

READS
370

4 authors:



Luciano Silva
Federal University from Jequitinhonha and Mucuri's Valleys
1 PUBLICATION **3** CITATIONS
SEE PROFILE



Marcelo Ferreira Rego
Federal University of Ouro Preto
9 PUBLICATIONS **30** CITATIONS
SEE PROFILE



Luciana Pereira de Assis
Federal University from Jequitinhonha and Mucuri's Valleys
91 PUBLICATIONS **202** CITATIONS
SEE PROFILE



Alessandro Vivas Andrade
Federal University from Jequitinhonha and Mucuri's Valleys
94 PUBLICATIONS **211** CITATIONS
SEE PROFILE

All content following this page was uploaded by Luciano Silva on 23 July 2019.

The user has requested enhancement of the downloaded file.

Algorithms for the Unrelated Parallel Machine Scheduling Problem with Sequence Dependent Setup Times

Luciano Geraldo Silva
Open and Distance Education Directorate
Federal University of Jequitinhonha and Mucuri Valleys
Diamantina, Brazil
luciano.tj2011@gmail.com

Marcelo Ferreira Rego
Computer Department
Federal University of Ouro Preto
Ouro Preto, Brazil
marcelofr@gmail.com

Luciana Pereira de Assis
Computer Department
Federal University of Jequitinhonha and Mucuri Valleys
Diamantina, Brazil
lupassis@gmail.com

Alessandro Vivas Andrade
Computer Department
Federal University of Jequitinhonha and Mucuri Valleys
Diamantina, Brazil
prof.alessandrovivas@gmail.com

***Abstract*—This work addresses the Unrelated Parallel Machines Scheduling Problem with Sequence-Dependent Setup Times - UPMSPST. The objective considered here is to minimize the scheduling maximum completion time, also known as *makespan*. This is important in the practical sense as the UPMSPST is widely found in industries. As well, in a theoretical sense, it belongs to the NP-Hard class. Five algorithms have been implemented to find solutions for the UPMSPST. The results obtained by the execution of these algorithms were compared. Among all the algorithms studied here, the VNS and F&O heuristic with the VNS metaheuristic generating the initial solution presented the best results. In some cases, it even presented better results than in [1].**

***Index Terms*—unrelated parallel machines scheduling, UPMSPST, metaheuristic, mathematical heuristics, neighborhood structures, VNS, genetic algorithm, fix-and-optimize, relax-and-fix, gurobi**

I. Introduction

This article approaches the Unrelated Parallel Machines Scheduling Problem with Sequence-Dependent Setup Times - UPMSPST. In this problem, there are a set of jobs that need to be processed in one individual machine out of a set of parallel machines available. The problem consists of choosing the best machine, and the best sequence for the execution of each job, so that the *makespan* (maximum completion time) be the least amount of time possible.

Five algorithms were implemented to find good solutions for the UPMSPST. Instances provided in [1], and available for download in [2], were used to test and compare each algorithm. The first one is an exact method based on the C++ API from the solver Gurobi. The second one is the Variable Neighborhood Search (VNS) method, which consists of a metaheuristic that uses the concept of neighborhood structures to find better solutions and escape the local optimum. Then, a Genetic Algorithm (GA), an optimization method based on the natural evolution process, was adapted for the UPMSPST. Finally, two heuristics based on the mathematical modeling called Relax-and-Fix (R&F), and Fix-and-Optimize (F&O) were developed. They were built applying the concept of partitioning the problems' variables, then solving the partitioned sub-problems using the solver Gurobi.

978-1-5386-9233-2/18/$31.00 ©2018 IEEE

II. Problem Description

In this problem, there is a set $N = \{1, \ldots, n\}$ of jobs and a set $M = \{1, \ldots, , m\}$ of machines. It has the following properties:

(i) each job j, $j \in N$, must be processed exactly once and only on one machine i, $i \in M$;
(ii) there is a processing time P_{ij}, for each job j allocated at a machine i, and it depends on which machine the job was allocated;
(iii) between the processing of a job j and other k, there is a setup time S_{ijk}, that representing the time needed by the machine i to be ready to process a job k after job j has been processed at the same machine;
(iv) the setup time for the first job is represented by S_{i0k} with 0 denoting a dummy job that precedes a regular job k that will be processed on machine i;

A. Mathematical Model

The UPMSPST was modeled by [1] through the Mixed Integer Programming (MIP), and it is defined as follows:

M : set of unrelated machines
N : set of independent jobs

p_{ij} : processing time of job j at machine i
S_{ijk} : setup time on machine i when job k is processed soon after job j had been processed at the same machine

This model has the following decision variables:

$$X_{ijk} : \begin{cases} 1, & \text{if job } j \text{ precedes job } k \text{ at machine } i \\ 0, & \text{otherwise} \end{cases}$$

C_{ij} : completion time of job j at machine i
C_{max} : $makespan$, maximum completion time

The objective of the problem is the minimization of the *makespan*, maximum completion time, and it is given by the function:

$$min \quad C_{max} \tag{1}$$

The constraints of the model are:

$$\sum_{i \in M} \sum_{\substack{j \in \{0\} \cup \{N\} \\ j \neq k}} X_{ijk} = 1, \qquad \forall k \in N, \tag{2}$$

$$\sum_{i \in M} \sum_{\substack{k \in N \\ j \neq k}} X_{ijk} \leq 1, \qquad \forall j \in N, \tag{3}$$

$$\sum_{k \in N} X_{i0k} \leq 1, \qquad \forall i \in M, \tag{4}$$

$$\sum_{\substack{h \in \{0\} \cup \{N\} \\ h \neq k, h \neq j}} X_{ihj} \geq X_{ijk}, \quad \forall j, \quad k \in N, \quad j \neq k, \quad \forall i \in M, \tag{5}$$

$$C_{ik} + V(1 - X_{ijk}) \geq C_{ij} + S_{ijk} + p_{ik}, \quad \forall j \in \{0\} \cup \{N\}, \quad \forall k \in N, \quad j \neq k, \quad \forall i \in M, \tag{6}$$

$$C_{i0} = 0, \qquad \forall i \in M, \tag{7}$$

$$C_{ij} \geq 0, \qquad \forall j \in N, \quad \forall i \in M, \tag{8}$$

$$C_{max} \geq C_{ij}, \qquad \forall j \in N, \quad \forall i \in M, \tag{9}$$

$$X_{ijk} \in \{0, 1\}, \quad \forall j \in \{0\} \cup \{N\}, \quad \forall k \in N, \quad j \neq k, \quad \forall i \in M. \tag{10}$$

The constraint in Equation 2 assures that each job is allocated in one individual machine, with only one predecessor. The Equation 3 limits the maximum number of successors of each job. The restriction in Equation 4 does the same for the dummy jobs. The restriction in Equation 5 verifies the appropriate distribution for the jobs. It ensures that if a job j is processed at the machine i, then there is a job h preceding the job j in this same machine. The use of dummy job was necessary to control the completion times in each machine. Equation 6 defines the following: if a job k is allocated in a machine i after a job j ($X_{ijk} = 1$), then its completion time (C_{ik}) – should be bigger than the completion time of C_{ij} plus the processing time of k, and the setup time between j and k. In this case if $X_{ijk} = 0$, the constant V will make the Equation 6 redundant. For dummy jobs, the completion time is 0 according to restriction in Equation 7. Equation 8 ensures a non-negative completion time for regular jobs. The makespan is obtained in Equation 9. Finally, in Equation 10 specifies that the variables in the model are binary.

III. Literature Review

The UPMSPST can be found in several areas of science and also in the industrial context. The first works that undertook this problem appeared in the early 1950s' literature [3], and since then they have been the object of many studies. The UPMSPST is of practical importance, since it can be found in real applications such as the electronic industry [4], ironworks [5], textile industry [6] among others [7]. Additionally, it is theoretically important because it belongs to the set of *NP-Hard* class problems.

Based on a real case of scheduling problem at a Brazilian Company, [7] implemented the Greed Randomized Adaptive Search Procedure (GRASP), and Variable Neighborhood Search (VNS), in an attempt to minimize the sum of the *makespan* and the weighted tardiness. However, the problem considered in this work is different, since here the tardiness is not allowed. The VNS was the most efficient method for instances bigger than sixty jobs.

Two genetic algorithms, GA1 and GA2, were proposed by [1], differing from each other by the adopted parameters. These algorithms include a fast local search, and a local search enhanced crossover operator. The objective was to minimize the *makespan*. The algorithm GA2 obtained the best results. Moreover, CPLEX[1] was used to find optimum solutions for small instances. The instances applied to test the GA algorithms were created by the authors, and are available in [2].

[8] presented three hybrid heuristic algorithms with the goal of minimizing the *makespan*. The algorithms were: IVP, which combines the heuristics Iterated Local Search (ILS), Variable Neighborhood Descent (VND) and Path Relinking (PR); GIVP, that differs from the first one because it uses the Greed Randomized Adaptive Search Procedure (GRASP) to generate the initial solution; and GIVMP, which uses Mixed Relinking instead of Backward Relinking in the phase of Path Relinking, as well as a module of local search made by the CPLEX[1]. The author concluded that the GIVMP algorithm showed the best results. Moreover, his results were much better than the results presented by [1].

Four metaheuristics were implemented and evaluated by [9], with the goal of minimizing the *makespan*. The metaheuristics employed were the following: Simulated Annealing (SA), Iterated Local Search (ILS), Late Acceptance Hill Climbing (LAHC), and Step Counting Hill Climbing (SCHC). The best results were found with the use of the SA metaheuristic. These

[1] www.cplex.com

results were better than the results obtained by [1] and [8] for the large instances created by [1].

A hybrid local search approach, based on a metaheuristic and exact algorithm, is proposed by [10], with the goal of minimizing the makespan. The heuristic uses the solver CPLEX[1] to find neighbors in a *multi-start* algorithm.

IV. Methodology

This section describes how the algorithms were implemented. In this problem, the objective function assesses the *makespan* with the view to minimize it. A solution is an array of machines; each position has a list of jobs to be processed. Each machine has a completion time to run all the jobs allocated to it. The maximum completion time will be the *makespan* for the solution.

A. Metaheuristics

In this subsection, the metaheuristics implemented in this work are explained. The stopping criteria used for the heuristics in this work were the same ones used by [1]. Equation 11 shows how this calculation is done. n and m represents the number of jobs and machines, respectively, and the parameter t ranges between $10, 30,$ and 50.

$$stopExecution = n \times (m/2) \times t\ ms, \qquad t = \{10, 30, 50\} \quad (11)$$

1) Variable Neighborhood Search: According to [11], Variable Neighborhood Search (VNS) is a modern metaheuristic that uses the idea of exploring neighborhood changes systematically, thus it explores the search space without following a path but exploring distant neighborhoods [12]. The VNS uses the neighborhood structures strategy to explore the search space. A local search in the neighborhood is performed in order to find neighboring solutions to improve the current solution.

This algorithm requires three components: an initial solution, a local search method and strategies to generate the neighborhood structures. For the initial solution a greedy approach was used, in which the processing times of the jobs in each machine were evaluated to allocate the best job for each machine, thus generating a feasible solution for the problem. For the local search, the *First Improvement* heuristic was used. Its strategy is to interrupt the search right away once it finds a solution better than the current one. In the worst case, this heuristic will search the entire neighborhood. This strategy makes it possible to avoid the computational effort of investigating the entire neighborhood. Although it does not guarantee that the local optimum of the neighborhood will be found, it is possible to achieve good results. Finally, the neighborhood structures perform movements in a given solution s, and these motions will generate new neighborhood solutions of s. The neighborhood structures used here were taken from [9], and are described as follows:

- Task Move: The job j that is allocated in the processing sequence of a machine M_x will be moved to be processed in the scheduling of another machine M_y;
- Shift: A job j that is allocated in a certain machine processing sequence M_x will be shifted to a different position of same machine;
- Switch: The jobs j and k that are allocated in the machine processing sequence M_x will be switched from position to scheduling with one another. Thus, j assumes the position of k in the machine M_x and vice versa;
- Swap: A job j allocated on the machine M_x will swap position with the job k that is in another machine M_y. Thus, j will assume the position of k in the machine M_y, whereas k will assume the position of j in the machine M_x;
- 2-realloc: This structure is similar to the Shift structure, but it uses two jobs instead.

2) Genetic Algorithm: According to [13], Genetic Algorithm (GA) uses evolutionary concepts of nature, such as crossover, mutation and selection mechanism, to find solutions to a given problem. Genetic Algorithms are search and combinatorial optimization algorithms based on the natural selection conceived by Charles Robert Darwin. The natural selection states that the most adaptive generation remains while the least adaptive disappears with time. An individual's chromosomes are mixed with another individual's chromosomes to generate new individuals, that will constitute future generations. These descendants have the characteristics of the selected parents, and with the use of mutation in these descendants it is possible to achieve diversity in the new generations. This way, the individuals of the new generations are more evolved than those in previous generations. That is, with time, only the best individuals will continue to exist. The bad ones will not occur in the new generations.

The following items are the basis of the GA algorithm:

- Individual: is a solution for the problem;
- Initial Population: the population is a set of individuals, and its size is defined by a parameter P_{size}. The individuals present in the initial population are created from the *Multiple Insertion* (MI) heuristic which was also used by [1] and was proposed by [14]. This method constructs a solution to the problem addressed, selecting at each step a job that will be inserted into the solution, the insertion is performed in the machine and in the position that provides the smallest value for the objective function of the problem, *makespan*.
- Selection Operator: it is done through the tournament selection method, where t individuals are chosen randomly from the population and the best of them is the one chosen, that is, the one with the smallest *makespan*. This process is repeated until there is a new population.
- Crossover operator: *One Point Order Crossover* has been adapted to the problem of parallel machine sequencing. The use of this operator has the objective of generating new offspring in the population. It works as follows: you must first make a random selection of two individuals from the current population. Then a cut point p_i is set randomly in the sequencing of each machine i. Then, a

offspring is generated from the sequencing of jobs that are between the first and the position p_i of each machine i of the first individual, the sequence of the other jobs, follows the sequencing of the second individual, performing the insertion of each job missing in position that produces the smallest value for the *makespan*.

- Mutation Operator: it can be applied considering a certain probability P_m. The mutation applies the motion of Shift.

The algorithm starts with the initial population with P_{size} individuals, then two individuals are selected through Selection Operator. According to a given probability P_c , the Crossover Operator will be used in the chosen individuals, called parents, with the objective of generating two new individuals, called offspring. After that, the Mutation Operator can be applied to the offspring considering a certain probability P_m.

It is worth to mention that the generated offspring will only be added to the population if their *makespan* value is better than the one of the worst individual in the population. Otherwise, the generated child will be rejected and will not be inserted in the population.

Four variations of GA were proposed by [1]. The authors showed that the variation denoted by GAStd4, obtained the best results. For more details regarding the differences between each of the variations proposed by them, see [1]. The parameters of this variation GAStd4 are in Table Table I. The GA, as well as the GAStd4 variation methods implemented in this work adopted the same parameters specified by the authors.

Table I
VALUES OF PARAMETERS OF VARIATION GASTD4 BY [1] AFTER CALIBRATION

Parameter	Value
Population size (P_{size})	80
Pressure	10
Probability of Crossover (P_c)	0.5
Probability of Mutation (P_c)	0.5

B. Mathematical Heuristics

These heuristics are based on the mathematical modeling described in subsection II-A. They need a partitioning criterion in order to make subproblems of the original one. These partitioned subproblems are then solved using the *Gurobi* solver. The stopping criteria was the same used by [1] for the CPLEX solver that was an hour of execution time. The strategies of partitioning the problem's variables were the following:

- Machine Partitioning: the number of partitions is defined by the number of machines, where each partition is a machine. It is possible that the jobs of that partition are free to change their order in the execution sequence of machine jobs, which affects the completion time of that machine;
- Job Partitioning: each partition is a job. In this type of partitioning, the processing time and setup time are considered, because the job when changing the machine, the time the job spends to be processed, and the time of setup spent by the machine to be configured before performing this job are different on each machine;
- Machine and Job Partitioning: the two previous methods are fused into a single partitioning strategy. In this strategy it is possible to get both benefits because of the jobs changing positions on the same machine they are in, either switching to another machine that produces a shorter completion time result.

1) Relax-and-Fix: According to [15] the heuristic was described by [16] as a method that fragments the MIP model into submodels by partitioning the set of variables. Those submodels are fast solved because of the linear relaxation of the variables. Thus, it makes possible to solve the problem quickly, although the optimal solution for the original problem is not guaranteed.

This heuristic has been used in many combinatorial optimization problems, such as batch sizing problems ([17], [18] and [19]) and also in real problems in the industry ([20], [21], [22], [23] and [15]).

The variables of the problem are partitioned by a particular strategy. Initially, all partitions are relaxed. Then, for each partition performs the following: make the partition binary, solve the problem with a binary partition and the rest continuous. Finally, fix the selected partition. The partitions fixed by the process will no longer change. Algorithm 1 describes this method.

Algorithm 1: Relax-and-Fix – R&F

```
Let P be a set of partitions of the problem's binary variables;
Relax all the variables of all partitions;
for (partition i ∈ P) do
    Change i to be binary;
    Solve the problem with i binary and the rest relaxed;
    Fix the variables of partition i;
end
```

2) Fix-and-Optimize: According to [23] the heuristic was described by [24] as initially named as *exchange*, and later called *Fix-and-Optimize* in [25]. The heuristic goal is to find better quality solutions than the initial solution provided. This heuristic was applied in solving batch sizing problems ([26], [27], [25], [28] and [29]).

In this heuristic all the binary variables are fixed according to an initial solution, then dividing these variables into a set of partitions, with partitions defined, the heuristic performs the following steps: selects a partition from the partition set, changes that partition to be binary, solves the problem with a binary variable partition and the rest fixed, evaluates the given solution. This process is repeated in the next partition until all partitions go through this. The Algorithm 2 describe this process.

V. COMPUTATIONAL RESULTS

Two groups of instances were used to test the algorithms. The first group with 16 small instances, and the second with

Algorithm 2: Fix-and-Optimize – F&O

```
Given a viable solution of the problem;
Let P be a set of partitions of the problem's binary variables;
Fix all the variables of all partitions;
for (partition i ∈ P) do
    Change i to be binary;
    Solve the problem with i binary and the rest fixed;
    if (viable solution and better than current solution) then
        Fix the variables of partition i;
    end
    else
        Discard solution;
    end
end
```

25 large instances. Those instances were generated from each combination $n \times m$ in the restricted strip $1-9$. For the small instances the number of jobs varied among $n = \{6, 8, 10, 12\}$ and the number of machines varied among $m = \{2, 3, 4, 5\}$. For the large instances the number of jobs varied among $n = \{50, 100, 150, 200, 250\}$ and the number of machines varied among $m = \{10, 15, 20, 25, 30\}$. All the instances that were used are available in [2].

The metric Relative Percentage Deviation (RPD) was used for the comparison of the implemented algorithms. It is the same used by [1], and it is defined by Equation 12. In this equation, $Method_{sol}$ is the value of the objective function (*makespan*) for each instance of the implemented algorithm, and $Best_{sol}$ is the value of the *makespan* of the best-known solution in [2]. RPD determines the quality of a solution in relation to the best available results in [2]. Thus, the smaller the RPD value, the better the quality of the solution.

$$RPD = \frac{Method_{sol} - Best_{sol}}{Best_{sol}} \times 100 \tag{12}$$

Considering the Equation 12, it is possible to conclude that negative RPD values indicate that the solution found by the algorithm is better than the solution found in [2]. Therefore, positive values indicate the opposite.

The results are presented as follows: MIP are the values found by the exact method with the *Gurobi* API; VNS are the results of the metaheuristic *Variable Neighborhood Search*; GA refers to the values for the *Genetic Algorithm*; R&F are the results obtained by applying the math heuristic *Relax-and-Fix*; F&O VNS are the values for the math heuristic *Fix-and-Optimize* with the initial solution generated from the VNS; finally, F&O GA are the results obtained by the *Fix-and-Optimize* method combined with the use of a GA to obtain the initial solution.

Three executions were performed for each instance of the tested set, in each implemented algorithm. The best result obtained in these executions was considered and presented in the following tables.

The Tables II, III and IV present the results as follows: the first column indicates the name of the instance and the RPD values obtained as the result of the application of the algorithms in each instance from the set of small instances tested. In the group $n \times m$ of instances, the n is the number of jobs and m is the number of machines. In the other columns the results of each algorithm are presented, with 3 algorithm executions for each instance. The stopping criterion adopted in each algorithm is defined in Equation 11, where the parameter t can assume one of the following values: $10, 30$ or 50. Thus, each table presents the results considering a different value for t. It is important to note that these three values were also used by [1] on their proposed algorithms.

Table II
SMALL INSTANCES: RPD VALUE WHEN $t = 10$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
6×2	0.00	0.00	0.00	68.92	0.00	0.00
6×3	0.00	0.00	0.00	1.92	0.00	0.00
6×4	0.00	0.00	0.00	9.09	0.00	0.00
6×5	0.00	0.00	0.00	0.00	0.00	0.00
8×2	0.00	0.00	0.00	43.13	0.00	0.00
8×3	0.00	0.00	0.00	202.86	0.00	0.00
8×4	0.00	0.00	0.00	33.02	0.00	0.00
8×5	0.00	0.00	0.00	280.00	0.00	0.00
10×2	0.00	0.00	0.00	27.43	0.00	0.00
10×3	0.00	0.00	0.00	182.28	0.00	0.00
10×4	0.00	0.00	0.00	80.60	0.00	0.00
10×5	0.00	0.00	0.00	123.08	0.00	0.00
12×2	0.00	0.00	0.00	51.98	0.00	0.00
12×3	0.00	0.00	0.00	43.93	0.00	0.00
12×4	0.00	0.00	0.90	81.08	0.00	0.90
12×5	0.00	0.00	0.00	136.25	0.00	0.00
Average	0.00	0.00	0.06	85.35	0.00	0.06

As shown in Table II, the MIP, VNS, F&O VNS algorithms have obtained RPDs equal to zero. This means that they have found the best value for the instances of the problem. The GA and F&O GA algorithms failed to find the best value only for the instance 12×4. The R&F algorithm obtained the worst results.

Table III
SMALL INSTANCES: RPD VALUE WHEN $t = 30$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
6×2	0.00	0.00	0.00	68.92	0.00	0.00
6×3	0.00	0.00	0.00	1.92	0.00	0.00
6×4	0.00	0.00	0.00	9.09	0.00	0.00
6×5	0.00	0.00	0.00	0.00	0.00	0.00
8×2	0.00	0.00	0.00	43.13	0.00	0.00
8×3	0.00	0.00	0.00	202.86	0.00	0.00
8×4	0.00	0.00	0.00	33.02	0.00	0.00
8×5	0.00	0.00	0.00	280.00	0.00	0.00
10×2	0.00	0.00	0.00	27.43	0.00	0.00
10×3	0.00	0.00	0.00	182.28	0.00	0.00
10×4	0.00	0.00	0.00	80.60	0.00	0.00
10×5	0.00	0.00	0.00	123.08	0.00	0.00
12×2	0.00	0.00	0.00	51.98	0.00	0.00
12×3	0.00	0.00	0.00	43.93	0.00	0.00
12×4	0.00	0.00	2.53	81.08	0.00	2.53
12×5	0.00	0.00	0.00	136.25	0.00	0.00
Average	0.00	0.00	0.16	85.35	0.00	0.16

Table III had similar results as the Table II, except for the GA and F&O that presented worse results for the 12×4 instance.

Table IV
SMALL INSTANCES: RPD VALUE WHEN $t = 50$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
6×2	0.00	0.00	0.00	68.92	0.00	0.00
6×3	0.00	0.00	0.00	1.92	0.00	0.00
6×4	0.00	0.00	0.00	9.09	0.00	0.00
6×5	0.00	0.00	0.00	0.00	0.00	0.00
8×2	0.00	0.00	0.00	43.13	0.00	0.00
8×3	0.00	0.00	0.00	202.86	0.00	0.00
8×4	0.00	0.00	0.00	33.02	0.00	0.00
8×5	0.00	0.00	0.00	280.00	0.00	0.00
10×2	0.00	0.00	0.00	27.43	0.00	0.00
10×3	0.00	0.00	0.00	182.28	0.00	0.00
10×4	0.00	0.00	0.00	80.60	0.00	0.00
10×5	0.00	0.00	0.00	123.08	0.00	0.00
12×2	0.00	0.00	0.00	51.98	0.00	0.00
12×3	0.00	0.00	0.00	43.93	0.00	0.00
12×4	0.00	0.00	0.00	81.08	0.00	0.00
12×5	0.00	0.00	0.00	136.25	0.00	0.00
Average	0.00	0.00	0.00	85.35	0.00	0.00

Conforming to the Table IV only the mathematical heuristic R&F did not obtain good solutions for any instance meaning that this heuristic does not seem fitting for this kind of sequence problem. Nonetheless, the others algorithms achieved the best-known results for the small instances.

Table V
LARGE INSTANCES: RPD VALUE WHEN $t = 10$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
50×10	62.69	4.48	129.85	925.37	4.48	129.85
50×15	33.33	5.56	166.67	1141.67	5.56	166.67
50×20	29.03	9.68	103.23	1116.13	9.68	103.23
50×25	0.00	0.00	336.36	1940.91	0.00	336.36
50×30	0.00	18.18	681.82	1936.36	18.18	663.64
100×10	61.83	-6.11	151.91	501.81	-6.11	151.91
100×15	72.06	7.35	138.24	758.82	7.35	138.24
100×20	100.00	6.52	215.22	1773.91	6.52	213.04
100×25	–	6.45	251.61	–	6.45	251.61
100×30	–	27.59	224.14	–	27.59	224.14
150×10	–	4.35	167.39	–	4.35	165.76
150×15	–	3.09	205.15	–	3.09	152.58
150×20	–	20.00	240.00	–	20.00	226.67
150×25	–	8.33	206.25	–	8.33	206.25
150×30	–	5.56	327.78	–	5.56	327.78
200×10	–	5.77	145.38	–	5.77	145.38
200×15	–	10.69	153.44	–	10.69	153.44
200×20	–	10.26	180.77	–	10.26	180.77
200×25	–	14.75	101.64	–	14.75	101.64
200×30	–	6.12	232.65	–	6.12	232.65
250×10	–	11.15	138.85	–	11.15	138.85
250×15	–	6.67	160.00	–	6.67	160.00
250×20	–	17.00	257.00	–	17.00	257.00
250×25	–	11.84	209.21	–	11.84	209.21
250×30	–	22.81	222.81	–	22.81	222.81
Average	44.87	9.52	213.89	1261.87	9.52	210.38

The Table V shows that the VNS and F&O VNS gotten the same results for all the large instances, when $t = 10$; which means that using the F&O VNS was not worth in this case, because the VNS alone is faster than the mathematical heuristic combined with VNS. The value -6.11 of RPD for the instance 100×10 indicates that the algorithms found a solution better than the best solution ([2]). Also, the MIP achieved two results of RPD equal to zero, meaning that the exact method found the best-known solution for the instances 50×25 and 50×30. The algorithms GA, R&F and F&O GA do not found good solutions for those instances, because their values of RPD are too big, indicating that the solution found by them were too far from the best-known solution.

Tables fields with the dash (–), means that the solver *Gurobi* was not able to return a valid solution. That happens when the memory RAM is insufficient to execute the mathematical model, although the computer used in these tests had more than 20 GB of memory (RAM + swap).

Table VI
LARGE INSTANCES: RPD VALUE WHEN $t = 30$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
50×10	62.69	4.48	129.85	925.37	4.48	129.85
50×15	33.33	0.00	166.67	1141.67	0.00	166.67
50×20	29.03	6.45	103.23	1116.13	6.45	103.23
50×25	0.00	0.00	336.36	1940.91	0.00	318.18
50×30	0.00	0.00	654.55	1936.36	0.00	581.82
100×10	61.83	-6.11	119.08	501.81	-6.11	119.08
100×15	72.06	-1.47	138.24	758.82	-1.47	138.24
100×20	100.00	4.35	213.04	1773.91	4.35	213.04
100×25	–	9.68	251.61	–	9.68	251.61
100×30	–	17.24	224.14	–	17.24	224.14
150×10	–	2.72	169.02	–	2.72	169.02
150×15	–	2.06	215.46	–	2.06	215.46
150×20	–	13.33	242.67	–	13.33	242.67
150×25	–	0.00	206.25	–	0.00	206.25
150×30	–	5.56	330.56	–	5.56	330.56
200×10	–	2.69	145.38	–	2.31	145.38
200×15	–	6.87	154.20	–	6.87	152.67
200×20	–	3.85	179.49	–	3.85	179.49
200×25	–	9.84	101.64	–	9.84	101.64
200×30	–	6.12	232.65	–	6.12	232.65
250×10	–	6.83	137.77	–	6.83	137.77
250×15	–	4.67	152.00	–	4.67	152.00
250×20	–	12.00	257.00	–	12.00	257.00
250×25	–	6.58	209.21	–	6.58	209.21
250×30	–	15.79	221.05	–	15.79	221.05
Average	44.87	5.34	211.64	1261.87	5.33	207.95

The VNS and F&O VNS presented, in Table VI, better results than Table V when $t = 30$. For the instances 50×15, 50×25, 50×30 and 150×25 they obtained the best-known results because the RPD value is zero, moreover, they outperformed in the instances 100×10 and 100×15 getting negative RPD, which means a better result than the literature. The MIP and R&F acquired the same results as shown in Table V. A small improvement can be seen in the GA and F&O GA, but still, those results are unsatisfactory.

When $t = 50$ as shown in Table VII, the algorithms enhance results, except for MIP and R&F. Comparing VNS and F&O VNS that got the best results, these were almost the same, that shows how efficient is the VNS metaheuristic to find good solutions for this kind of sequence problem. For that reason the results of the VNS when $t = 50$, will be compared with the results presented by [1] in the Tables VIII and IX.

In order to compare the VNS with the results presented by [1], five executions were performed for each instance of

Table VII
LARGE INSTANCES: RPD VALUE WHEN $t = 50$

$n \times m$	Algorithm					
	MIP	VNS	GA	R&F	F&O VNS	F&O GA
50×10	62.69	4.48	71.05	925.37	4.48	60.53
50×15	33.33	0.00	166.67	1141.67	0.00	166.67
50×20	29.03	6.45	103.23	1116.13	6.45	103.23
50×25	0.00	0.00	264.86	1940.91	0.00	259.46
50×30	0.00	0.00	259.26	1936.36	0.00	248.15
100×10	61.83	-6.11	151.91	501.81	-6.11	142.75
100×15	72.06	-1.47	138.24	758.82	-1.47	138.24
100×20	100.00	0.00	215.22	1773.91	0.00	215.22
100×25	–	3.23	210.96	–	3.23	210.96
100×30	–	17.24	224.14	–	17.24	224.14
150×10	–	-2.72	111.45	–	-2.72	106.63
150×15	–	2.06	162.86	–	2.06	162.86
150×20	–	5.67	200.00	–	5.67	200.00
150×25	–	0.00	176.79	–	0.00	175.89
150×30	–	2.78	242.35	–	2.78	242.35
200×10	–	-1.20	139.57	–	-1.20	139.57
200×15	–	6.87	135.18	–	6.87	118.18
200×20	–	6.41	178.98	–	6.41	178.98
200×25	–	7.80	98.36	–	7.80	98.36
200×30	–	6.12	174.59	–	6.12	174.59
250×10	–	0.20	130.40	–	-0.20	130.40
250×15	–	0.32	158.67	–	0.00	158.67
250×20	–	12.00	134.05	–	12.00	134.05
250×25	–	6.58	169.71	–	6.58	169.71
250×30	–	15.79	197.83	–	15.79	197.83
Average	44.87	3.70	168.65	1261.87	3.67	166.30

Table VIII
SMALL INSTANCES: COMPARISON BETWEEN VNS AND THE BEST PRESENTED IN [1]

$n \times m$	VNS	[1]
6×2	0.00	0.00
6×3	0.00	0.08
6×4	0.00	0.37
6×5	0.00	0.12
8×2	0.00	0.00
8×3	0.00	0.31
8×4	0.00	0.41
8×5	0.00	0.11
10×2	0.00	0.07
10×3	0.00	0.18
10×4	0.00	0.30
10×5	0.00	1.03
12×2	0.00	0.10
12×3	0.00	0.12
12×4	0.00	0.89
12×5	0.00	1.49
Average	0.00	0.35

the tested set. The average of the results obtained in these executions was considered and presented in the following tables.

For the small instances, the Table VIII compares VNS and GA2 proposed by [1]. In all instances, the VNS beat the GA2. However, Table IX shows that the GA2 obtained better results then VNS for instances bigger than 200×25. Nevertheless, the VNS presented better results for the rest of the large instances, which means that the VNS proved to be a good algorithm to find good solutions for the UPMSPST with low computer costs.

Table IX
LARGE INSTANCES: COMPARISON BETWEEN VNS AND THE BEST PRESENTED IN [1]

$n \times m$	VNS	[1]
50×10	5.37	6.49
50×15	4.45	9.20
50×20	0.00	9.57
50×25	0.00	8.10
50×30	0.00	9.40
100×10	-3.21	5.54
100×15	2.28	7.32
100×20	-2.23	8.59
100×25	5.16	8.07
100×30	12.00	7.90
150×10	0.23	5.28
150×15	1.65	6.80
150×20	3.17.	7.40
150×25	-1.25	7.05
150×30	6.12	7.17
200×10	3.09	4.24
200×15	5.04	6.21
200×20	5.75	6.71
200×25	15.74	6.82
200×30	12.94	7.09
250×10	9.62	4.38
250×15	13.42	5.12
250×20	17.34	6.92
250×25	15.62	6.02
250×30	24.22	5.91
Average	6.26	6.93

VI. CONCLUSIONS

In this work the mathematical model, mathematical heuristics, and metaheuristics applied to UPMSP were evaluated. The *Gurobi* solver was used in the mathematical heuristics based on the mathematical modeling, and five neighborhood structures were used for the VNS metaheuristic. The same parameters presented by [1] were adopted for the *Genetic Algorithm* implemented in this work.

The VNS metaheuristic and the heuristic F&O using VNS to find the initial solution obtained the best results for the small instances, as well as for the large ones. It is relevant to highlight that it is necessary the use of a simple metaheuristic for VNS to be efficient and achieve good results for the problem, with low computer costs. Furthermore, the use of methods based on the mathematical modeling demonstrated to be very efficient when applied to NP-Hard class problems, as the UPMSP.

REFERENCES

[1] E. Vallada and R. Ruiz, "A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times," *European Journal of Operational Research*, vol. 211, no. 3, pp. 612–622, 2011. [Online]. Available: http://dx.doi.org/10.1016/j.ejor.2011.01.011

[2] G. de investigación SOA-ITI: Sistemas de Optimización Aplicada Instituto Tecnológico de Informática. (2013) SOA-ITI. [Online]. Available: http://soa.iti.es/problem-instances

[3] M. L. Pinedo, *Scheduling Theory, Algorithms, and Systems*, 2008.

[4] D. Kim, K. Kim, W. Jang, and F. Frank Chen, "Unrelated parallel machine scheduling with setup times using simulated annealing," *Robotics and Computer-Integrated Manufacturing*, vol. 18, no. 3-4, pp. 223–231, 6 2002.

[5] L. Tang and X. Wang, "Simultaneously scheduling multiple turns for steel color-coating production," *European Journal of Operational Research*, vol. 198, no. 3, pp. 715–725, 2009. [Online]. Available: https://doi.org/10.1016/j.ejor.2008.09.025

[6] M. Gendreau, G. Laporte, and E. M. Guimarães, "A divide and merge heuristic for the multiprocessor scheduling problem with sequence dependent setup times," *European Journal of Operational Research*, vol. 133, pp. 183–189, 2001.

[7] M. G. Ravetti, "Algoritmos para o Problema de Seqüenciamento com Máquinas Paralelas e Tempos de Preparação Dependentes da Seqüência," 2007.

[8] M. N. Haddad, "Algoritmos Heurísticos Híbridos para o Problema de Sequenciamento em Máquinas Paralelas Não-Relacionadas com Tempos de PReparação Dependentes da Sequência," p. 88, 2012.

[9] C. L. T. d. F. e. Silva, "Metaheurísticas de Busca Local para o Problema de Sequenciamento de Tarefas em Máquinas Paralelas Não Relacionadas com Tempo de Preparação Dependente da Sequência," 2014.

[10] F. Stefanello, O. B. Araújo, F. M. Muller, and M. Zanetti, "MIP–Based Neighborhood Search for the Unrelated Parallel Machine Scheduling Problem With Sequence and Machine-Dependent Setup Times," *International Journal of Advanced Manufacturing Technology*, 2015.

[11] P. Hansen and N. Mladenovic, *A tutorial on variable neighborhood search*. Groupe d'études et de recherche en analyse des décisions, HEC Montréal, 2003.

[12] P. Hansen and N. Mladenović, "Variable neighborhood search: Principles and applications," *European journal of operational research*, vol. 130, no. 3, pp. 449–467, 2001.

[13] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*, 3rd ed. Springer-Verlag Berlin Heidelberg, 1996.

[14] M. E. Kurz and R. G. Askin, "Heuristic scheduling of parallel machines with sequence-dependent set-up times," *International Journal of Production Research*, vol. 39, no. 16, pp. 3747–3769, 2001. [Online]. Available: http://dx.doi.org/10.1080/00207540110064938

[15] J. D. de Oliveira, "ESTRATÉGIAS RELAX-AND-FIX APLICADA AO PROBLEMA DE ROTEAMENTO EM ARCOS CAPACITADO E PERIÓDICO," p. 98, 2017.

[16] L. A. Wolsey, *Integer Programming*, 1998.

[17] G. Belvaux and L. A. Wolsey, "bc — prod: A specialized branch-and-cut system for lot-sizing problems," *Management Science*, vol. 46, no. 5, pp. 724–738, 2000. [Online]. Available: https://doi.org/10.1287/mnsc.46.5.724.12048

[18] C. MercÉ and G. Fontan, "Mip-based heuristics for capacitated lotsizing problems," *International Journal of Production Economics*, vol. 85, no. 1, pp. 97 – 111, 2003, planning and Control of Productive Systems. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0925527303000902

[19] K. Akartunali and A. J. Miller, "A heuristic approach for big bucket multi-level production planning problems," *European Journal of Operational Research*, vol. 193, no. 2, pp. 396 – 411, 2009. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0377221707011307

[20] D. Ferreira, R. Morabito, and S. Rangel, "Relax and fix heuristics to solve one-stage one-machine lot-scheduling models for small-scale soft drink plants," *Computers Operations Research*, vol. 37, no. 4, pp. 684 – 691, 2010. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0305054809001695

[21] S. A. de Araujo, M. N. Arenales, and A. R. Clark, "Lot sizing and furnace scheduling in small foundries," *Computers Operations Research*, vol. 35, no. 3, pp. 916 – 932, 2008, part Special Issue: New Trends in Locational Analysis. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0305054806001390

[22] M. O. Santos and B. Almada-Lobo, "Integrated pulp and paper mill planning and scheduling," *Computers Industrial Engineering*, vol. 63, no. 1, pp. 1 – 12, 2012. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0360835212000204

[23] A. L. da Cunha, "Métodos heurísticos para um problema de planejamento da produção em uma indústria química," 2013.

[24] Y. Pochet and L. A. Wolsey, *Production planning by mixed integer programming*. Springer Science & Business Media, 2006.

[25] F. Sahling, L. Buschkühl, H. Tempelmeier, and S. Helber, "Solving a multi-level capacitated lot sizing problem with multi-period setup carry-over via a fix-and-optimize heuristic," *Computers & Operations Research*, vol. 36, no. 9, pp. 2546–2553, 2009.

[26] R. J. James and B. Almada-Lobo, "Single and parallel machine capacitated lotsizing and scheduling: New iterative mip-based neighborhood search heuristics," *Computers & Operations Research*, vol. 38, no. 12, pp. 1816–1825, 2011.

[27] J. C. Lang and Z.-J. M. Shen, "Fix-and-optimize heuristics for capacitated lot-sizing with sequence-dependent setups and substitutions," *European Journal of Operational Research*, vol. 214, no. 3, pp. 595–605, 2011.

[28] S. Helber and F. Sahling, "A fix-and-optimize approach for the multi-level capacitated lot sizing problem," *International Journal of Production Economics*, vol. 123, no. 2, pp. 247–256, 2010.

[29] C. F. M. Toledo, M. da Silva Arantes, M. Y. B. Hossomi, P. M. França, and K. Akartunalı, "A relax-and-fix with fix-and-optimize heuristic applied to multi-level lot-sizing problems," *Journal of Heuristics*, vol. 21, no. 5, pp. 687–717, 2015.

View publication stats